🎓 Aula 4 – Desvendando o Jest (Testar como um Dev Profissional)

🔥 8. Quebrando o código (aprendizado real)

```
export function dividir(a: number, b: number): number {
    if (b === 0) {
        throw new Error('Divisão por zero’);
    }
    return a / b;
}
```

“Agora complicou 😱, será?”

```
import { dividir } from '../src/dividir’;

describe('Função dividir', () => {

    it('deve dividir corretamente', () => {

      expect(dividir(10, 2)).toBe(5);

    });

    it('deve lançar erro ao dividir por zero', () => {

      expect(() => dividir(10, 0)).toThrow();

    });

});
```

🧠 Evoluindo para padrão profissional

Uma estrutura inicial comum seria assim:

📄 Código base

‹/› TypeScript

```typescript
export function calcularDesconto(valor: number): number {
  if (valor > 100) {
    return valor * 0.9;
  }
  return valor;
}
```

🧪 Teste básico (antes)

</> TypeScript

```
expect(calcularDesconto(200)).toBe(180);
```

🧠 Evoluindo para padrão profissional

```
import { calcularDesconto } from '../src/calcularDesconto';

describe('Função calcularDesconto', () => {

  it('deve aplicar desconto quando valor for maior que 100', () => {
    expect(calcularDesconto(200)).toBe(180);
  });

  it('não deve aplicar desconto quando valor for menor ou igual a 100', () => {
    expect(calcularDesconto(100)).toBe(100);
  });

});
```

🧠 6. Explicando Matchers (com prática)

🔹 toBe (valor simples)

TypeScript

```typescript
expect(2 + 2).toBe(4);
```

🔹 toEqual (objetos)

TypeScript

```typescript
expect({ nome: 'Eder' }).toEqual({ nome: 'Eder' });
```

🧠 6. Explicando Matchers (com prática)

🔹 toThrow (erro)

‹/› TypeScript

```typescript
expect(() => dividir(10, 0)).toThrow();
```

🔹 toBeTruthy / toBeFalsy

‹/› TypeScript

```typescript
expect(true).toBeTruthy();
expect(false).toBeFalsy();
```

🧠 Insight

"Escolher o matcher certo evita erro falso."

🧪 7. Prática Guiada Forte

Criar função

‹/› TypeScript

```typescript
export function ehPar(numero: number): boolean {
  return numero % 2 === 0;
}
```

Criar testes completos

```
import { ehPar } from '../src/ehPar';

describe('Função ehPar', () => {

  it('deve retornar true para número par', () => {
    expect(ehPar(4)).toBeTruthy();
  });

  it('deve retornar false para número ímpar', () => {
    expect(ehPar(3)).toBeFalsy();
  });

});
```

🧠 Boas práticas

- Nome claro no describe
- Nome claro no it
- Não misturar cenários
- Testes independentes

Exercício Individual

Criar testes para:

```typescript
function validarIdade(idade: number): boolean
```

Regras:

maior ou igual a 18 → true

menor que 18 → false

🧠 Momento Ouro

“Se eu leio seu teste e não entendo o sistema...o teste falhou.”

🧠 Fechamento

“Hoje você saiu de alguém que escreve teste...
para alguém que escreve teste profissional.”

Obrigado!

Prof. Éder Silva

Senac